Deep Hooks: Monitoring native execution in WoW64 applications – Part 2

March 19, 2018 (https://www.sentinelone.com/blog/2018/03/)
by Yarden Shafir & Assaf Carlsbad (https://www.sentinelone.com/blog/author/carlsbad/)

Where we left off

In the first part (https://www.sentinelone.com/blog/deep-hooks-monitoring-native-execution-wow64-applications-part-1/) of this series we presented several injection methods capable of injecting 64-bit DLLs into WoW64 (https://www.sentinelone.com/blog/now-stage-deep-hooks-monitoring-native-execution-wow64-applications/) processes, with the intention to eventually use this DLL to hook 64-bit API functions in the process.

We finished the post by presenting injection via APC, and saw that, when tested to CFG-aware processes, it failed to inject the DLL and crashed the process. To understand why that happens, we have to dive into some of the implementation details of CFG.

A brief introduction to CFG

CFG (**C**ontrol **F**low **G**uard) is a relatively new exploit mitigation feature, first introduced in Windows 8.1 Update 3 and later enhanced in Windows 10. It is a compiler-enabled mitigation meant to combat memory corruption vulnerabilities by preventing indirect calls to non-legitimate targets. Right before every indirect function call, the compiler inserts an additional call to a dedicated validation routine located in NTDLL. This routine receives the call target and checks whether or not it is the start address of a function, within 8 bytes of granularity. If it is not, a security check failure (int 0×29) is raised and the process is forcibly terminated.

```
; int kuhl_m_sekurlsa_clean(...)
kuhl_m_sekurlsa_clean proc near
sub     rsp, 28h
mov     rax, cs:lsassLocalHelper
call    qword ptr [rax+8]
add     rsp, 28h
retn
kuhl_m_sekurlsa_clean endp
```

```
; int kuhl_m_sekurlsa_clean(...)
kuhl_m_sekurlsa_clean proc near

cleanLocalLib= qword ptr -18h

sub     rsp, 38h
mov     rax, cs:lsassLocalHelper
mov     rax, [rax+8]
mov     [rsp+38h+cleanLocalLib], rax
mov     rcx, [rsp+38h+cleanLocalLib]
call    cs:__guard_check_icall_fptr
call    [rsp+38h+cleanLocalLib]
add     rsp, 38h
retn
kuhl_m_sekurlsa_clean endp
```

Figure 8 – Mimikatz, compiled with (right) and without (left) CFG.

To make this validation easy and efficient, CFG utilizes a new memory region added especially for that purpose, called a CFG bitmap. In this bitmap, each bit represents the status of 8 bytes in the process' address space and marks whether or not they constitute a valid call target. Because of this mapping ratio, the bitmap has to be 1/64 of the total virtual address space of the process, which can get quite large – 2TB in 64-bit processes, whose total address space is 128TB.

Obviously, in 64-bit processes most of this bitmap is uncommitted, as only a very small portion of the process' address space is actually in use. Only when a new executable page is introduced (either by directly allocating virtual memory, mapping a view of a

section object or otherwise changing page protection to executable), the kernel commits and then sets the bits corresponding to that page in the bitmap.

CFG in WoW64 processes

In his blog post (http://www.alex-ionescu.com/?p=300), titled 'Closing "Heaven's Gate"', Alex Ionescu described some of the unique characteristics of CFG in WoW64 processes. As shown there, CFG-aware WoW64 processes have not one, but two separate CFG bitmaps:

- A native bitmap, marking valid call targets for 64-bit code in the process. Since this bitmap has to be inaccessible to 32-bit code, it resides above the 4GB boundary, usually next to the native NTDLL.
- A WoW64 bitmap, marking valid call targets for 32-bit code in the process. Its reserved size is 32MB as it only covers the lower 4GB of the address space, where 32-bit code can live. Obviously, it is always located below the 4GB boundary, usually next to the main image.

Since WoW64 processes have two CFG bitmaps and two versions of NTDLL loaded into them, there are, naturally, two versions of the validation function as well. The 32-bit version of the function checks the supplied address against the WoW64 bitmap, while the 64-bit version checks it against the native bitmap.

```
0: kd> !vad ffffd30450a71220
VAD           Level     Start       End Commit
ffffd3045bfce8f0  4       760       761      2 Private      READWRITE
ffffd3045c191870  3       770       788      0 Mapped       READONLY           Pagefile section, shared commit 0
ffffd30453804170  4       790       7cf     11 Private      READWRITE
ffffd304529f95d0  2       7d0       7d3      0 Mapped       READONLY           Pagefile section, shared commit 0
ffffd30453f681d0  3       7e0       7e2      0 Mapped       READONLY           Pagefile section, shared commit 0
ffffd304651a7df0  4       7f0       7f0      1 Private      READWRITE
ffffd30460b28310  1       800       9ff      5 Private      READWRITE
ffffd30458f08110  3       a00       a3f     19 Private      READWRITE
ffffd3045dc344d0  2       b00       b0f      4 Private      READWRITE
ffffd3045133e170  3      11f0      122e      3 Mapped  Exe  EXECUTE_WRITECOPY  \Windows\SysWOW64\notepad.exe
ffffd30450a71220  0      1230      322f      6 Mapped       NO ACCESS          Pagefile section, shared commit 0
ffffd304622071a0  4      3230      5230      1 Private      READWRITE
ffffd304639d24d0  3     52580     525d0      3 Mapped  Exe  EXECUTE_WRITECOPY  \Windows\System32\wow64.dll
ffffd30463449570  4     525e0     52655      4 Mapped  Exe  EXECUTE_WRITECOPY  \Windows\System32\wow64win.dll
ffffd30457957620  2     77760     778ec      7 Mapped  Exe  EXECUTE_WRITECOPY  \Windows\SysWOW64\ntdll.dll
ffffd304583ae310  3     7f180     7f1a2      0 Mapped       READONLY           Pagefile section, shared commit 0
ffffd3045b6c1ad0  1     7ffe0     7ffe0      1 Private      READONLY
ffffd30457db44e0  3     7ffe1     7ffef     -1 Private      READONLY
ffffd3045db15350  4     7fff0 7df9feb3f     -1 Private      READONLY  RESERVED
ffffd30458c54280  2 7df9feb40 7ff9feb3f      9 Mapped       NO ACCESS          Pagefile section, shared commit 0
ffffd3045fd8d120  3 7ff9feb40 7ff9fed1f     13 Mapped  Exe  EXECUTE_WRITECOPY  \Windows\System32\ntdll.dll
ffffd304600a79a0  4 7ff9fed20 7fffffef     -1 Private      READONLY  RESERVED
```

32 bit bitmap

32 bit ntdll

64 bit bitmap

64 bit ntdll

Figure 9 – a snapshot of the virtual address space of a 32-bit notepad.exe, running on Windows 10 x64

As mentioned earlier, the kernel sets bits in the CFG bitmap whenever a new executable page is introduced. This raises the question, which of the two bitmaps is affected in the case of WoW64 processes? As Ionescu pointed out, the answer to that lies in the *MiSelectCfgBitMap()* and *MiSelectBitMapForImage()* functions, called by the memory manager whenever a change to the CFG bitmap is required.

```
kd> k
 # Child-SP          RetAddr           Call Site
00 ffffd001`18dc6668 fffff803`a3137169 nt!MiSelectBitMapForImage
01 ffffd001`18dc6670 fffff803`a2d29ebb nt!MiSelectCfgBitMap+0x45
02 ffffd001`18dc66a0 fffff803`a3137787 nt!MiCommitVadCfgBits+0x103
03 ffffd001`18dc66e0 fffff803`a3084c2d nt!MiMapViewOfImageSection+0x617
04 ffffd001`18dc6830 fffff803`a308a7e1 nt!MiMapViewOfSection+0x33d
05 ffffd001`18dc69a0 fffff803`a2dd5263 nt!NtMapViewOfSection+0x2e1
06 ffffd001`18dc6a90 00007ffd`39a537ca nt!KiSystemServiceCopyEnd+0x13
```

Figure 10 – Partial call stack showing the call to MiSelectCfgBitMap() and MiSelectBitMapForImage() when executable memory is mapped into a process

The pseudocode of these two functions is presented below:

```
PVOID * MiSelectCfgBitmap(_In_ PEPROCESS CurrentProcess, _In_ PVOID Address, _In_Opt_ PSEGMENT Segment)
{
    if ((CurrentProcess->WoW64Process != NULL) &&
        ((CurrentProcess->WoW64Process.Machine == IMAGE_FILE_MACHINE_I386) ||
            (CurrentProcess->WoW64Process.Machine == IMAGE_FILE_MACHINE_ARMNT)) &&
        (Address < _4gb) && ((Segment == NULL) || (MiSelectBitMapForImage(Segment) == BynamicBaseBitMap32)))
    {
        return KeGetCurrentPrcb()->CurrentThread.ApcState.Process.Vm.Instance.VmWorkingSetList.Wow64CfgBitmap;
    }
    else
    {
        return KeGetCurrentPrcb()->CurrentThread.ApcState.Process.Vm.Instance.VmWorkingSetList.NativeCfgBitmap;
    }
}
```

Figure 11.1 – pseudocode of MiSelectCfgBitMap() as seen on Windows 10 x64 RS3

```
MI_DYNAMICBASE_BITMAP MiSelectBitMapForImage(_In_ PSEGMENT Segment)
{
    if (FlagOn(Segment->SegmentFlags, Binary32))
    {
        return DynamicBaseBitMap32;
    }

    if ((FlagOn(Segment->ImageInformation.ExportedImageInformation.ImageFlags, BaseBelow4gb)) &&
        (FlagOn(Segment->ImageInformation.ExportedImageInformation.ImageCharacteristics, IMAGE_FILE_DLL)))
    {
        return DynamicBaseBitMapWow64Dll;
    }

    else
    {
        if (Segment->BaseAddress < _4gb)
        {
            return DynamicBaseBitMap64Low;
        }
        else
        {
            return DynamicBaseBitMap64High;
        }
    }
}
```

Figure 11.2 – pseudocode of MiSelectBitMapForImage() as seen on Windows 10 x64 RS3

A few conclusions can be drawn from these two functions:

1. As can be expected, all 32-bit modules are marked in the WoW64 bitmap.
2. All 64-bit modules are marked in the native bitmap, including those which are mapped into the lower 4GB of the address space. This is essential, as otherwise the native NTDLL would not be able to interoperate with the native DLLs comprising the WoW64 environment. For example, NTDLL would not even be able

to load these modules as the call to their entry point would fail the CFG validation logic.

3. All private memory allocations below the 4GB boundary are marked in the WoW64 bitmap, regardless of who allocated them or for what purpose. As the astute reader has probably noticed already, in the example shown in Figure 8 all of the address space above the 4GB boundary is reserved (with the exception of the native NTDLL and the native CFG bitmap). Since memory cannot be allocated from reserved regions, this effectively means that **all** private memory allocations will be marked exclusively in the WoW64 bitmap.

It now becomes clear why the previously shown technique for DLL injection using APC is bound to fail: although the “adapter thunk” contains 64-bit code, it is a private memory allocation and as such it will populate the WoW64 bitmap. However, the function responsible for the initial dispatching of APCs is the 64-bit version of *KiUserApcDispatcher()*, which will attempt to validate the thunk’s address against the native bitmap, but to no avail.

So, if we wish to maintain our APC injection capabilities, we must somehow modify our technique to overcome the CFG validation problem.

Back to APC injection

Having some prior knowledge regarding CFG implementation details, one might suggest to simply mark the adapter thunk as a valid call target by calling *NtSetInformationVirtualMemory()* with the *VmCfgCallTargetInformation* information class. This option, albeit promising, won't actually solve the problem. The reason for that being that internally, *NtSetInformationVirtualMemory()* relies on *MiSelectCfgBitMap()* to help decide which of the two bitmaps should be affected. For the same reasons described earlier, *MiSelectCfgBitmap()* will still return the WoW64 bitmap when supplied with the address of the adapter thunk, thus leaving the native bitmap untouched.

Alex Ionescu
@aionescu

Follow

Cute... APC injection into Windows 10 Wow64 processes breaks when CFG is enabled (very subtle bitmap selection issue)

4:57 PM - 11 Dec 2015

10 Retweets **12** Likes

5 10 12

Tweet your reply

R0bert R0senb0rg @drProct0r · 11 Dec 2015

Replying to @aionescu

@aionescu Problem is, that there is an API to disable CFG for a given memory region, but it seems that is does not work properly

1

Alex Ionescu @aionescu · 12 Dec 2015

@drProct0r It suffers from the same problem if you try to use it in this context.

1

R0bert R0senb0rg @drProct0r · 14 Dec 2015

@aionescu just to make sure,you are talking about APC before WoW is initialized right?

initialized right?

1

Alex Ionescu @aionescu · 14 Dec 2015

@drProct0r shouldn't matter. the issue is that on Windows 10, the CFG bitmap for the lower 4GB is stored in one place, and > 4GB in another

1

Alex Ionescu @aionescu · 14 Dec 2015

@drProct0r so if you allocate an APC from kernel that ends up at < 4GB and then try to execute it (in 64-bit mode), you'll get a CFG failure

1

R0bert R0senb0rg @drProct0r · 15 Dec 2015

@aionescu So NtSetInformationVirtualMemory will set the wrong bitmap?I can't allocate my APC in >4GB for WoW proc, so is there any solution?

1

Alex Ionescu @aionescu · 16 Dec 2015

@drProct0r 32-bit APC :) I think this deserves a blog post! From the Maldives.

Lemma 12 – NtSetInformationVirtualMemory() will only affect the WoW64 bitmap as it internally relies on MiSelectCfgBitMap(). Proof: Alex Ionescu says so. Q.E.D

After disqualifying this solution, the next option that comes into mind is finding a way to somehow “trick” *MiSelectCfgBitmap()* into returning the native bitmap, right when memory for the adapter thunk is allocated.

“Nativising” a WoW64 process

When looking at the pseudocode of *MiSelectCfgBitmap()* presented in figure 11.1, it is clearly visible that the native bitmap will always be returned for “true” 64-bit processes. This is obvious, as 64-bit processes should only have a single, native CFG bitmap. Therefore, if we somehow manage to “nativise” the WoW64 process, the adapter thunk will be marked in the native bitmap and so the APC dispatching should succeed as planned.

The kernel’s way of telling whether a given process is a native one or not is by probing the *WoW64Process* member of the *EPROCESS* structure. If this member is set to NULL, the process is considered to be a native one, otherwise it is treated as a WoW64 process.

```
0:000> dt nt!_EPROCESS
ntdll!_EPROCESS
   +0x000 Pcb              : _KPROCESS
   +0x2d8 ProcessLock      : _EX_PUSH_LOCK
   ...
   +0x420 DebugPort        : Ptr64 Void
   +0x428 WoW64Process     : Ptr64 _EWOW64PROCESS
   +0x430 DeviceMap        : Ptr64 Void
   ...
```

Figure 12 – a (very) partial view of the EPROCESS structure as seen in Windows 10 RS3. Notice the Wow64Process pointer at offset 0×428.

With that in mind, we can apply a DKOM-based solution, in which *WoW64Process* is zeroed-out right before allocating memory for the adapter thunk and restored to its original value afterwards.

```
originalWow64Process = CurrentProcess->WoW64Process;
CurrentProcess->WoW64Process = NULL;
ZwAllocateVirtualMemory(ZwCurrentProcess(), ..., PAGE_EXECUTE_READWRITE);
CurrentProcess->WoW64Process = originalWow64Process;
```

Figure 13 – pseudocode for making the adapter thunk occupy the native bitmap

This solution, presented in Appendix B, makes our APC injection succeed in CFG-aware WoW64 processes, and was tested on Windows 10 RS3.

Although simple, this method does have some significant downsides. First, the *EPROCESS* structure that has to be modified is largely undocumented and changes often (http://terminus.rewolf.pl/terminus/structures/ntdll/_EPROCESS_combined.html)

between Windows releases. Therefore, the offset of *WoW64Process* inside that structure cannot be relied on to remain constant and has to be searched for heuristically during runtime. Second, zeroing out the *WoW64Process* member can have some unexpected side effects and dangers, especially in cases where the process contains several threads.

To conclude, this is a valid option for making the APC injector work in CFG-aware processes, but it is rather unstable and unreliable, and should be used with extreme caution. Taking these downsides into consideration, we wanted to find a more reliable solution to the problem, preferably one that does not rely on private, executable memory allocations at all.

Thunkless APC injection

When initializing an APC, it is possible to set the APC routine to point to any function of our choice, whether it is an existing function or one created by us especially for that purpose. This means that – theoretically at least – we can inject a DLL by creating an APC that will directly call the native *LdrLoadDll()*, without going through the adapter thunk at all. Obviously, *LdrLoadDll()* is a valid call target for 64-bit code, and thus it can serve as an APC target without triggering a CFG violation.

However, at the binary level there seems to be a problem: the prototypes of *LdrLoadDll()* and *KNORMAL_ROUTINE* don't match up. While *LdrLoadDll()* expects four arguments, functions of type *KNORMAL_ROUTINE* only seem to receive three:

```
NTSTATUS
LdrLoadDll (
        _In_ PWCHAR PathToFile,
        _In_ ULONG Flags,
        _In_ PUNICODE_STRING ModuleFileName,
        _Out_ PHANDLE ModuleHandle
        );
```

Figure 14.1 – the prototype of LdrLoadDll()

```
VOID
KNORMAL_ROUTINE (
    _In_opt_ PVOID NormalContext,
    _In_opt_ PVOID SystemArgument1,
    _In_opt_ PVOID SystemArgument2
    );
```

Figure 14.2 – the prototype of KNORMAL_ROUTINE

Still, one should consider the *__fastcall* calling convention used in accordance with the x64 ABI: the first four arguments of every function are passed to it via registers RCX, RDX, R8 and R9, and so when *LdrLoadDll()* will be called by *KiUserApcDispatcher(),* whatever value is currently held by R9 will be interpreted as the fourth parameter. According to the prototype presented above, the fourth parameter received by *LdrLoadDll()* is declared as *"_Out_ PHANDLE ModuleHandle"*. This means that for *LdrLoadDll()* to succeed, R9 must contain a valid pointer to a writable memory location capable of holding pointer-sized data.

Unfortunately, as the standard APC procedure takes only three parameters, there is obviously no way to specify a value for a fourth one during APC initialization time. As a result, the value held by R9 upon entry to the APC routine is basically unknown. So the question arises: can we somehow guarantee that R9 will hold a valid pointer such that it satisfies all of *LdrLoadDll()* requirements? Surprisingly enough, the answer to this question is positive, but how can we be sure of that?

```
KERNELAPI
VOID
KeInitializeApc (
    _Out_ PRKAPC Apc,
    _In_ PRKTHREAD Thread,
    _In_ KAPC_ENVIRONMENT Environment,
    _In_ PKKERNEL_ROUTINE KernelRoutine,
    _In_opt_ PKRUNDOWN_ROUTINE RundownRoutine,
    _In_opt_ PKNORMAL_ROUTINE NormalRoutine,
    _In_opt_ KPROCESSOR_MODE ApcMode,
    _In_opt_ PVOID NormalContext
    );
```

```
BOOLEAN
NTAPI
KeInsertQueueApc (
    _Inout_ PRKAPC Apc,
    _In_opt_ PVOID SystemArgument1,
    _In_opt_ PVOID SystemArgument2,
    _In_ KPRIORITY Increment
    );
```

Figure 15 – prototypes of KeInitializeApc and KeInsertQueueApc. The user-controlled parameters for the user-mode APC routine (NormalRoutine) are highlighted in red.

In his post exploring some of the internal aspects of APC dispatching, Skywing (http://www.nynaeve.net/?p=202) demonstrated that the 64-bit *KiUserApcDispatcher()* actually sends the APC routine a fourth, “hidden” argument, pointing to a *CONTEXT* structure. This structure holds the CPU state that is to be restored via *NtContinue()* when the APC dispatching process is finished. Although this post is rather old, looking at the implementation of *KiUserApcDispatcher()* in newer systems such as Windows 10 shows that this still holds:

```
; Exported entry 103. KiUserApcDispatcher


; Attributes: noreturn

public KiUserApcDispatcher
KiUserApcDispatcher proc near

sysarg1= qword ptr  8
sysarg2= qword ptr  10h
NormalRoutine= qword ptr  18h

mov     rcx, [rsp+NormalRoutine]
mov     rax, rcx          ; NormalRoutine
mov     r9, rsp           ; &context
sar     rcx, 2
mov     rdx, [rsp+sysarg1]
neg     rcx
mov     r8, [rsp+sysarg2]
shld    rcx, rcx, 20h
test    ecx, ecx
jz      short wow64Apc
```

```
mov     rcx, [rsp+0]      ; NormalContext
call    KiUserCallForwarder
```

```
wow64Apc:
mov     eax, [rsp+0]
or      rcx, rax
mov     rax, cs:Wow64ApcRoutine
test    rax, rax
jz      short loc_1800A8F9E
```

```
loc_1800A8F9E:
mov     rcx, rsp
mov     dl, 1
call    ZwContinue
test    eax, eax
jz      short KiUserApcDispatcher
```

Figure 16 – Part of the implementation of KiUserApcDispatcher() from native NTDLL in Windows 10 RS3. Notice that RSP, pointing to the CONTEXT structure, is moved into R9.

So, we can conclude that in this scenario, the value received by *LdrLoadDll()* as *ModuleHandle* will always point to a writable memory block which holds a *CONTEXT* structure, thus allowing for a successful injection. However, overwriting members of the *CONTEXT* structure might get risky; if any important information is trashed, the thread might crash when attempting to resume its execution after the call to *NtContinue()* is made. As we've seen before, *LdrLoadDll()* only writes 8 bytes (pointer size on x64) to the memory location pointed to by *ModuleHandle,* so it would only overwrite the first member of the CONTEXT structure, which happens to be *P1Home*:

```
0:000> dt nt!_CONTEXT
ntdll!_CONTEXT
   +0x000 P1Home           : Uint8B
   +0x008 P2Home           : Uint8B
   +0x010 P3Home           : Uint8B
   +0x018 P4Home           : Uint8B
   +0x020 P5Home           : Uint8B
   +0x028 P6Home           : Uint8B
   +0x030 ContextFlags     : Uint4B
   +0x034 ...
```

Figure 17 – the first 0×34 bytes of the CONTEXT structure. The parameters passed to the APC routine are stored in offsets 0, 0×8 and 0×10 of that context, while the address of the APC routine is at offset 0×18.

Luckily, the first four members of the *CONTEXT* structure are actually used to store the arguments (https://github.com/svn2github/reactos/blob/d461c3f6a3cb7ce06d1d63a3370556f71d305b21/ntoskrnl/ke/amd64/stubs.c#L242) for *KiUserApcDispatcher()* and are no longer required once the APC routine itself is executed. In order to make sure that overwriting *P1Home* is indeed safe, it is enough to take a look at the prolog of *KiUserApcDispatcher()*, presented in Figure 16. By carefully reviewing its prolog, we can see that *KiUserApcDispatcher()* has a somewhat unique calling convention. The top of the stack points to the aforementioned *CONTEXT* structure, which – In addition to a CPU state – also encapsulates the address of the APC routine and the values of the other three parameters that will be passed to it.

By correlating the offsets of this structure, shown in Figure 17, with the arguments' offsets presented in Figure 16, we can conclude that:

- P1Home holds *NormalContext*
- P2Home holds *sysarg1*
- P3Home holds *sysarg2*
- P4Home holds *NormalRoutine,* which is the address of the APC routine that will be called from *KiUserCallForwarder().*

```
0:001> dt nt!_CONTEXT @r9
ntdll!_CONTEXT
   +0x000 P1Home           : 0
   +0x008 P2Home           : 0
   +0x010 P3Home           : 0xee0000
   +0x018 P4Home           : 0x00007ffa`7da95af0
   +0x020 P5Home           : 0
   +0x028 P6Home           : 0
   +0x030 ContextFlags     : 0x10001b
   +0x034 ...
```

Figure 18 – the first 0×30 bytes of the CONTEXT structure when the APC routine is called

Since members P1Home to P4Home were never used to hold any CPU-related data, they will not be used by NtContinue() to restore the context. Knowing that, we can assume there is no harm in overwriting P1Home from the APC routine. We can now recreate our injector (shown in Appendix C) to inject a native module into any WoW64 process by queueing an APC that directly calls LdrLoadDll(), without causing the notorious CFG violation error.

Conclusion

This brings to an end the second part of the series. In these first two posts, we demonstrated the ability to inject 64-bit DLLs into WoW64 processes using several different methods. Obviously, more methods exist for doing so, but finding them is left as an exercise to the interested reader.

Up next: adapting an x64 hooking engine to support hooking the native NTDLL.

Appendixes

Appendix B – nativising a WoW64 process

```
#define scanSize 0x500  // in Windows 10 RS3, WoW64Process is at offset 0x428

PVOID GetEprocessWow64ProcessPtr(_In_ PEPROCESS Process)
{
    PAGED_CODE();
    NT_ASSERT(Process != NULL);
    PVOID wow64Info = PsGetProcessWow64Process(Process);
    if (wow64Info == NULL) {
        return NULL;
    }

    // Scan the EPROCESS structure to locate the pointer to WOW64 info that we have previously obtained.
    PCHAR currentPointer = (PCHAR)Process;
    for (ULONG index = 0; index < scanSize; currentPointer++, index++) {
        if (!MmIsAddressValid(currentPointer)) {
            return NULL;
        }

        PULONG_PTR ptr = (PULONG_PTR)(*(PULONG_PTR)currentPointer);
        if (!MmIsAddressValid(ptr)) {
            continue;
        }

        ULONG_PTR candidate = *ptr;
        if (candidate == (ULONG_PTR)wow64Info) {
            return currentPointer;
        }
    }
    return NULL;
}

void InjectByApc(_In_ PKTHREAD thread)
{
    ...
    __try {
        ...
        PVOID wow64ProcessAddress = GetEprocessWow64ProcessPtr(pEprocess);
        if (wow64ProcessAddress == NULL) {
            __leave;
        }
        ULONG_PTR originalWow64Process = *(PULONG_PTR)wow64ProcessAddress;
        // Disable kernel mode APCs delivery to the current thread.
        KeEnterGuardedRegion();

        *(PULONG_PTR)wow64ProcessAddress = 0;

        status = ZwAllocateVirtualMemory(ZwCurrentProcess(), (PVOID*)&UserAPC_mem, NULL, &apcsize,
MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
        if (!NT_SUCCESS(status) || (UserAPC_mem == NULL)) {
            __leave;
        }
```

```
        // And restore the WOWProcess in the EPROCESS back to its original value. That's it, we have
nativised the process :)
        *(PULONG_PTR)wow64ProcessAddress = originalWow64Process;

        // Re-enable kernel APCs.
        KeLeaveGuardedRegion();
        ...
    }

    __finally {
        KeUnstackDetachProcess(&apcState);
    }
    ...
}
```

Appendix C – thunkless APC injection

```
typedef NTSYSAPI NTSTATUS (NTAPI * LdrLoadDll_t)(
    _In_opt_ PWCHAR PathToFile,
    _In_opt_ ULONG Flags,
    _In_ PUNICODE_STRING ModuleFileName,
    _Out_ PVOID *ModuleHandle
);

typedef struct _INJECTION_CONTEXT
{
    UNICODE_STRING DllName;
    WCHAR Buffer[MAX_PATH];
} INJECTION_CONTEXT, *PINJECTION_CONTEXT;


VOID KernelApcRoutine(_In_ PVOID context, _In_ PVOID arg1, _In_ PVOID arg2, _In_ PVOID arg3, _In_ PVOID
arg4)
{
    UNREFERENCED_PARAMETER(arg1);
    UNREFERENCED_PARAMETER(arg2);
    UNREFERENCED_PARAMETER(arg3);
    UNREFERENCED_PARAMETER(arg4);

    // Free the context.
    ExFreePool(context);
}

BOOLEAN InjectDllByApc(_In_ PKTHREAD pTargetThread)
{
    PEPROCESS pTargetProcess  = NULL;
    PKAPC pKapcObj = NULL;
    KAPC_STATE apcState = { 0 };
    NTSTATUS status = STATUS_SUCCESS;
    PINJECTION_CONTEXT ctx = NULL;
    SIZE_T ctxSize = sizeof(INJECTION_CONTEXT);
    LdrLoadDll_t pLdrLoadDll = NULL;

    // Obtain the containing process.
    pTargetProcess = IoThreadToProcess(pTargetThread);
    if (pTargetProcess == NULL) {
        return FALSE;
    }

    // Is it a WoW64 process?
    if (PsGetProcessWow64Process(pTargetProcess) == NULL) {
        return FALSE;
    }

    // Allocate memory for the KAPC object.
```

```
    pKapcObj = (PKAPC)ExAllocatePool(PagedPool, sizeof(KAPC));
    if (pKapcObj == NULL) {
        return FALSE;
    }

    // Attach the current thread to the target process.
    KeStackAttachProcess(pTargetProcess, &apcState);

    __try {
        // Allocate memory for the APC context in the target process.
        status = ZwAllocateVirtualMemory(ZwCurrentProcess(), (PVOID *)&ctx, 0, &ctxSize, MEM_COMMIT |
MEM_RESERVE, PAGE_READWRITE);
        if (!NT_SUCCESS(status)) {
            __leave;
        }

        pLdrLoadDll = (LdrLoadDll_t)(GetExportedSymbolAddr(g_NtdllBase, "LdrLoadDll"));
        if (pLdrLoadDll == NULL) {
            __leave;
        }

        // Initialize the name of the DLL to inject.
        RtlInitEmptyUnicodeString(&ctx->DllName, ctx->Buffer, sizeof(ctx->Buffer));
        RtlUnicodeStringCopyString(&ctx->DllName, INJECTED_DLL_NAME);
    }
    __finally {
        KeUnstackDetachProcess(&apcState);
    }

    if ( (pKapcObj != NULL) && (ctx != NULL) && (pLdrLoadDll != NULL) ) {
        KeInitializeApc(pKapcObj, pTargetThread, OriginalApcEnvironment,
(PKKERNEL_ROUTINE)KernelApcRoutine, NULL, (PKNORMAL_ROUTINE)pLdrLoadDll, UserMode, NULL);
        return KeInsertQueueApc(pKapcObj, NULL, ctx, 0);
    }

    return FALSE;
}
```